Robustness Evaluation of QMIX In Partially Observable Path-Finding With Non-Monotonic Tradeoffs

Dawson Berry, Mustafa Dursunoglu
COMP4900I/COMP5801G
Prof . Zinovi Rabinovich

December 2025

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Motivation 3
2.2 Problem Statement 3
2.3 Project Goals 4

3 Background **5**
3.1 MARL and QMIX 5
3.2 Related Work 6
3.3 Baselines 6

4 Model **7**
4.1 Problem Formulation 7
4.2 Environment Definition 7
4.3 Reward structure 8
4.4 QMIX Implementation 10
4.5 Variable Team Sizes and Masking 11

5 Experiments **12**
5.1 Experimental Setup and Training 12
5.2 Baselines 12
5.3 Results 12

6 Conclusion **15**

1 Abstract

This report evaluates how robust QMIX is when deployed in conditions that break specific common MARL assumptions about fixed team sizes and invariant environment structure. We do this by using a custom partially observable maze-navigation task with spring traps that create non-monotonic tradeoffs. We implement QMIX with centralized training/decentralized execution and add padding/masking so a policy trained with K agents can be tested with $K - n$ agents, running cross-testing between two independently generated mazes to measure within-maze and cross-maze generalization. This report finds QMIX in this environment is highly sensitive to reward structure and that performance and transfer are largely unreliable, with failures plausibly tied to QMIX's fixed-structure mixing assumptions, distribution shift under padding, and difficulties learning trap-related behaviors under partial observability.

2 Introduction

Multi-agent reinforcement learning (MARL) has become a prominent area of research, driven by the increasing deployment of autonomous systems in complex, real-world environments. Whether in robotics, drone coordination, traffic management, or distributed sensor networks, many modern systems rely on teams of agents that must collaborate and adapt to changing conditions. However, despite its rising development in the sector, MARL still faces essential challenges that prevent the seamless integration of trained models into real operating scenarios. One of the most critical gaps is the assumption that the number of agents present during deployment matches the number used during training. In real applications, this assumption may not hold. Agents may malfunction, lose connectivity, deplete their power supply, lose communication or be reassigned, resulting in a team whose size changes unpredictably over time.

This discrepancy between laboratory assumptions and real-world conditions provides the foundation for our project. We investigate the robustness of QMIX, one of the widely implemented cooperative MARL algorithms, when exposed to variations in team size and environment structure. According to Samvelyan et al. there was a lack of standardized benchmarks for research and evaluation [1]. QMIX behavior when team size changes has not been extensively studied. Yet in real deployments, such variations are expected.

We examine the more realistic scenario in which a policy trained with K agents must operate effectively with $K - n$ agents. We examine robustness, generalization, and the fundamental representational limits of value-factorization methods. We designed a custom partially observable maze environment where agents navigate a partially observable maze, containing traps as a hidden dangers. The environment is intentionally designed to expose weaknesses in QMIX's monotonic guarantee and its mixing network.

2.1 Motivation

Consider a fleet of delivery drones operating across a city. Some drones may complete tasks faster than others, return to charging stations, or face unexpected weather conditions that temporarily take them offline. In a search-and-rescue mission, robots may suffer mechanical failures, lose connection to the central controller, or become stuck in debris. The system must remain functional even when its composition changes.

MARL algorithms such as QMIX assume a fixed number of cooperating agents throughout training and deployment. This assumption is deeply embedded in the design of their neural architectures, the dimensionality of the state and observation spaces, and the structure of the value decomposition networks. this raises questions about dynamic team-size scenarios, which are the norm rather than the exception outside the laboratory.

From a practical standpoint, the ability for a trained model to adapt to different team sizes or different environments, such as transfer learning, would bring substantial benefits. Instead of having to retrain a separate model for every deployment condition it would be far more useful if a single learned policy could transfer its knowledge across these variations. This motivates our interest in examining whether QMIX can handle such transfer or whether its value-decomposition structure limits this capability.

Another key motivation comes from the structure of decision making in cooperative systems. In many tasks, especially navigation or pursuit-evasion scenarios, the team must balance competing priorities such as minimizing travel time, maximizing survival, and distributing risk among agents. This metric curves trade-off inspired the design of our custom maze environment, where agents must navigate through potentially dangerous edges that may contain traps.

QMIX's monotonic mixing constraint is not especially well studied. QMIX enforces that the global team value increases monotonically with each agent's individual value estimate. However, the team may benefit when an individual agent performs a locally disadvantageous action—such as triggering a trap to make the path safe for others. Can a monotonic value decomposition algorithm properly learn behaviors that require asymmetric sacrifice or strategic risk allocation?

2.2 Problem Statement

We examine whether cooperative policies generalize across different team configurations. If performance collapses when few agents are removed, the algorithm may not be considered deployable in real-world settings. Fundamentally this

also examines if the value-decomposition and mixer network in QMIX generalize to state-space distribution shifts caused by dynamic team sizes in non-static environments. The architecture must implicitly learn to ignore or reinterpret absent-agent information, even though the model was never exposed to such scenarios during training.

We consider a setting where both the environment and the team size differ between training and evaluation. Two maze structures are generated independently using a maze generation algorithm, with varied trap placements. QMIX policies are trained on both mazes using the full team of K agents and a reduced team of $K - n$ agents. These policies are then cross-tested across mazes and team sizes. This structure allows us to evaluate within-maze and cross-maze generalization, and team-size transfer.

If QMIX fails to generalize across these conditions, what does this imply for its reliability in real-world cooperative tasks? Is the failure due to representational limitations, training instability, lack of robustness, or a fundamental assumption violation?

2.3 Project Goals

We aimed to investigate the robustness of QMIX under the aforementioned stressors. This was done by designing a custom environment that would force QMIX to generalize gracefully to agent loss, and make non-monotonic tradeoffs to achieve its goal.

Our environment exposes weaknesses in algorithms that rely on fixed structures or full observability. Our multi-agent maze environment is partially observable and contains traps. The goal at the center of the maze provides rewards proportional to the number of remaining agents.

The second step of this work was to implement QMIX in a way that allowed experimentation across team sizes. This includes action masking, reward structure and how QMIX reacts, and ensuring the system remains stable when some number of agents are changed. Tuning the algorithm and the environment such that the algorithm could learn effectively comprises the majority of the work done.

The third step of this study aims to evaluate generalization. The evaluation cases were:

- Within-maze evaluation with equal team size.
- Within-maze evaluation with reduced team size.
- Cross-maze evaluation with equal team size.

- Cross-maze evaluation with reduced team size.
- Cross-maze evaluation combined with team-size reduction

These evaluation categories allow us to identify whether QMIX struggles primarily with structural changes, or team-size variation.

3 Background

This section reviews the core ideas, algorithms, and prior work that informed the design of this research.

3.1 MARL and QMIX

Cooperative MARL involves multiple agents that must act together to achieve a shared objective. A well-known difficulty in this setting is the issue of non-stationarity: independent Q-learning treats other agents as part of the environment, causing the transition dynamics to shift continuously as each agent updates its policy. This makes learning unstable, especially when many agents are involved.

Another challenge arises with fully centralized critics. Although centralized critics can, in principle, evaluate joint actions effectively, they scale poorly because the joint action space grows exponentially with the number of agents. This becomes impractical even for moderate team sizes and is incompatible with scenarios where each agent must make decisions using only local information.

QMIX was introduced to address these limitations. Instead of learning one global value function, QMIX assigns each agent its own utility function $Q_i(\tau_i, a_i)$, where τ_i is the agent's local action–observation history. These utilities are combined by a central mixing network:

$$Q_{\text{tot}} = f(Q_1, \ldots, Q_n \mid s),$$

where s is a global state available only during training. The mixing network follows a monotonicity constraint,

$$\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0,$$

which guarantees that selecting greedy actions independently still yields a joint action that is optimal under the learned Q_{tot}. This allows QMIX to implement centralized training with decentralized execution (CTDE): global information can guide learning, while each agent uses only its own observations at test time.

QMIX inherits structural assumptions that make it sensitive to fixed team sizes. The mixing network expects contributions from a fixed number of agents, and

the monotonic decomposition limits the kinds of coordination patterns it can represent.

3.2 Related Work

Samvelyan et al. introduce the StarCraft Multi-Agent Challenge (SMAC) specifically to address the lack of standardized, challenging benchmarks for cooperative MARL under partial observability [1]. Their PyMARL framework includes QMIX as a primary baseline and demonstrates that value-decomposition methods can learn sophisticated policies. We adopt a similar CTDE setup with per-agent utilities combined by a monotonic mixing network, but place QMIX in a much more MAPF-like navigation scenario.

Stern et al. provide a systematic treatment of MAPF, introducing a unified terminology for assumptions (conflict definitions, agent-at-target behavior, objective functions) and emphasizing how inconsistent modeling choices have historically made it difficult to compare algorithms or even identify which papers are relevant for a given application. They also propose standardized grid-based MAPF benchmarks and show that these can still challenge search-based solvers[2]. Our maze environment borrows this MAPF scenario but does away with the collision constraints.

Davydov et al. prove that QMIX can perform pathfinding in partially observable grid environments. They consider agents with limited field of view navigating grids with obstacles and other agents, and show that a QMIX-style architecture can learn decentralized policies that coordinate to avoid collisions and reach goals, scaling to larger numbers of agents and map sizes [3]. This is by far the most relevant work and motivation showing feasibility of our experiment. We add spring traps that induce non-monotonic sacrifice trade-offs and explicitly test robustness to changes in maze topology and team size, where the monotonic factorization and training-testing distribution mismatch become critical failure modes.

3.3 Baselines

Independent DQN or random policies do not explicitly model coordination or adapt to different team sizes, which makes them unsuitable for understanding how cooperation changes when agents leave or fail. Similarly, classical shortest-path or MAPF planners assume full observability, static safe edges, and a fixed number of agents. These methods do not model learning under partial observability, cannot adapt to hidden traps, and do not capture cases where agents die or where the environment changes dynamically.

Because of these limitations, there is a clear need for a MARL algorithm that can learn coordinated behavior under partial observability, dynamic hazards, and uncertain edge conditions. QMIX provides this capability while still being

tractable to train and analyze. Its structure also allows us to investigate an important open question: how well can value-decomposition MARL methods generalize when team size decreases and when the environment differs from the one used during training?

4 Model

We used a vanilla implementation of QMIX and made our own modifications namely masking for the mixing network which will be discussed shortly. We used Wilsons algorithm and our own trap maker function to construct the environment.

4.1 Problem Formulation

The state was split into two pieces, a global state, and a local agent observation in the partially observable maze. The global state consisted of:

- All agents (X, Y) positions on the $m \times m$ grid.
- Alive/Arrived flags per agent.
- Start and Goal locations being $(0, 0)$, and $(\lfloor \frac{m}{2} \rfloor, \lfloor \frac{m}{2} \rfloor)$ respectively.

With a per agent local partially observable state consisting of:

- For each direction $N/E/S/W$ is there a maze edge?
- Has this edge been cleared (was a trap, now safe)?
- One-hot recent action

With per agent actions being discrete and selected from a length 5 vector of (North, East, South, West, Stay).

Multiple agents are allowed on any cell of the maze and make one move per step of the episode. Edges are trapped and if the agent traverses an edge that is trapped the agent dies, is masked off, and that edge is now marked as cleared and safe. As of current implementation the traps are only visible after they have sprung, so agents cannot directly see traps. The goal is to reach the center of the maze and maximize the probability of reaching the goal, and the number of surviving agents.

4.2 Environment Definition

The maze is currently generated using Wilson's algorithm which builds a connectivity graph between all cells on an $m \times m$ grid with exactly one path $S \to G$. We intended for there to be multiple paths and an algorithm like Kruskal's with cycle allowance where you allow merging of sets if they are already connected

but with a low probability. The goal is at the center of the maze as mentioned sitting at $(\lfloor \frac{m}{2} \rfloor, \lfloor \frac{m}{2} \rfloor)$, and all agents start at the start cell at $(0, 0)$. Traps are deployed in the maze with the TrapConfig() function with mean probability and half-width; per-edge Bernoulli draws. Traps reset between episodes and cleared edges are tracked for observations. We had intended to actually regenerate traps on the same maze each episode but for initial training and what we considered a simpler scenario traps were static for the entirety of training and testing per generated maze.

Episodes currently terminate when all agents reach the goal and as a last minute fix the most up to date code implements an all arrival requirement but forces arrived agents to continually take the "stay" action. This was deemed a good compromise but felt heavy handed. With the initial implementation we had designed the environment to conclude only after all agents reach the goal but this presented several issues. Episodes where all agents were required to reach the goal often ended up being entirely unproductive for both agents that did reach the goal, and agents who did not. Agents who reached the goal were very likely to reach the goal particularly early in the 250 step episode, almost always $\leq$ 100 steps in. For the remaining $\sim$ 150 steps goal reaching agents would wander away from the goal cycling around the center of the maze and occasionally into some far corners. Agents who did not reach the goal naturally spent the entire episode on a particularly unproductive random walk. Because QMIX presents a joint reward Q_{tot} to the entire team, episodes where the goal was reached and many agents were still alive were seen as highly productive episodes. This made training incredibly noisy for agents who did reach the goal as well as agents who did not reach the goal, any productive actions towards goal reaching always happened at the start of the episode and a great deal of the episode was then spent achieving nothing but rewarding it all the same. With the change to any agent arriving ending the episode the causality between reaching the goal and the reward was much stronger but this still presented issues where if agents never saw the goal on their initial random walks in early episodes they never had pressure to reach the goal in later episodes. Only agents who reached the goal early on in training with high ϵ values would be likely to continue goal reaching in later episodes as ϵ decayed and the policy was locked in. Essentially agents who never had a good random walk early were just carried by agents who did and received good reward.

4.3 Reward structure

Designing a reward structure was very tricky. The reward had to prefer trajectories that (1) reach the goal, (2) keep as many agents alive as possible, but (3) allow for self-sacrifice. Initially the environment provides a single shared team reward at each step composed of three main parts:

- A small negative step cost to discourage useless wandering.
- A negative reward for dying to a trap.

- Large positive reward for reaching the goal.

The idea was that agents shouldn't spend a lot of time doing nothing, shouldn't fling themselves into traps, and get to the goal. This initial reward structure turned out to be very poor in design and had abysmal performance. Between the negative step cost, and negative reward for stepping in a not-so-uncommon trap the agents basically learned to cycle around near the start cell and accomplish nothing since the best strategy really was to just avoid movement. Front loading the negative reward onto directly tying a cost to dying to a trap was a bad design even if the intention was to force policies that kept the largest number of agents alive. Even after making the step cost 0 and death to trap reward 0 agents still made nearly no progress since the goal was hit so rarely there was no sharp credit assignment distinguishing good walks from bad ones. To fix this we implemented a distance based oracle to shape movement reward. The BFS based distance potential function is:

$$\Phi(s) = -\sum_i d(pos_i, goal)$$

and is defined over the global state. The shaped reward adds $\gamma\Phi(s_{t+1}) - \Phi(s_t)$ to the base team reward. Essentially the closer the agents move to the center of the maze the stronger the reward. This acts like a beacon pulling agents in. In principle, this preserves a still optimal policy while accelerating learning since even partial feedback yields positive feedback. The full per step reward is computed by:

$$r_t = -\underbrace{c_{\text{step}}}_{\text{step cost}} - \underbrace{c_{\text{death}} D_t}_{\text{death penalty per new death}} + \underbrace{\gamma\,\Phi(s_{t+1}) - \Phi(s_t)}_{\text{potential-based shaping}} + \underbrace{G\,A_t\left(1 + \frac{S_t}{K}\right)}_{\text{goal reward with survivor bonus}}$$

Where if the episode has length T ($steps \quad t = 0, ..., T-1$) the undiscounted reward is:

$$R_{\text{episode}} = \sum_{t=0}^{T-1} r_t.$$

and the discounted reward that is estimated by the Q-function is:

$$G_0 = \sum_{t=0}^{T-1} \gamma^t\, r_t,$$

Where $\gamma \in (0, 1)$ is the discount factor. To comment on the distance based oracle we are unsure if the oracle is too heavy-handed and "cheating" but given the other difficulties of training and QMIX's intrinsically long training time this was one of the few reprieves we had. Coming back to traps now that movement was largely dealt with the concept of traps creating tradeoff between locally bad but globally good was still a sheer cliff for the purposes of training. The

contention between the mixing network and local utility networks in:

$$\underset{u}{\operatorname{argmax}}\, Q_{tot}(\tau, u) = \begin{pmatrix} argmax_{u_1} Q_1(\tau_1, u_1) \\ . \\ . \\ . \\ argmax_{u_n} Q_n(\tau_n, u_n) \end{pmatrix}$$

means the mixer must find a Q value that satisfies the environmental artifact of traps being simultaneously required for goal reaching (large positive reward), and traps being unneeded and leading to dead ends with no goal (no significant reward). This appears to be the main showstopper for traps and this isn't just about the mixer disagreeing with the agents tendencies for local action since this can in fact be driven in a particular direction by large Q_{tot}, nor is it about the non-monotonic tradeoffs since again a local agents policy can be driven by the globally optimum policy for which this lower utility action supports. This requirement where a single fixed Q must be learned where it supports self-sacrifice driven trap tripping that actually leads to good reward while also dealing with the potential of traps that lead nowhere is almost unreconcilable. In fact the only reason why a reasonable policy could have been learned at all is that we do not change the distribution of traps between episodes. This static trap layout at maze generation would allow a policy to effectively learn per trap behavior while overfitting dramatically to the specific training maze. If traps were redistributed between episodes just from the mathematical guarantees of QMIX it seems an entirely lost cause to deal with traps in our naive way. Ideally traps could initialize as edges with some small negative reward and slowly heat up in cost to traverse but then a similar issue of encouraging no movement occurs. This balance is very fragile and we have so far failed to find a good one.

4.4 QMIX Implementation

We used an implementation that closely followed the original value-decomposition framework. We use three components: A shared per-agent utility network Q_i, a monotonic mixing network Q_{tot} and corresponding target networks for stable temporal-difference learning. The configuration for these is found in the QMIX-Config structure. The agent network is implemented as a recurrent GRU-based module (AgentRNN), that maps each agents observation and hidden state to a vector of Q values over the discrete action vector $(N, E, S, W, Stay)$. Recurrence here is important since the environment is partially observable the GRU allows each agent to integrate information over time and partially reconstruct its own picture of the latent maze structure from its trajectory.

The per-agent utilities are combined by the MixerNet which is the namesake of QMIX and enforces its monotonicity constraint. The mixer takes the global state and the vector of per-agent Q_i values and outputs a single scalar $Q_{tot}(s, a)$. The mixers weights are generated by small hypernetworks conditioned on the

state and return non-negative mixing weights via ReLU to guarantee:

$$\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0,$$

This allows the joint action-value to depend on the global state while preserving monotonicity over Q_i, essentially justifying greedy action selection. Target copies of both the agent and mixer networks are maintained and updated periodically. Training minimizes the MSE TD error between Q_{tot} and a bootstrapped target built from the target networks and the next-state value.

QMIXLearner trains on full episodes in the EpisodeBuffer. For each episode the learner rolls out a trajectory using epsilon-greedy per-agent decisions. After each joint action, the environment returns the next observations, centralized state, team reward, done flag, and updated availability, where these are packed into a transition object and appended to the episode. Once there are enough episodes in the buffer the learner samples mini-batches of entire episodes and computes loss.

4.5 Variable Team Sizes and Masking

Resizing of the mixer network is impossible to do dynamically, at least, with the skill level and time constraints we had. Instead when we need to kill an agent with a trap or when we remove n agents for executing the $K - n$ tests we mask and pad. This keeps all the networks happy and they are still instantiated for a fixed maximum number of K agents. Observations and availability vectors are padded with n dummy entries up to K during the $K - n$ tests, again keeping the mixer network happy. Dummy and dead agents are constrained to the STAY action and marked as not alive. During training, an alivemask tensor indicates, for each time step and agent, whether that agent is active. This mask is used to zero out Q values for the inactive agents before they are fed into the mixer, and prevent those utilities from receiving gradients. The QMIX learner effectively operates on a fixed-size agent set while the environment sees the correct variable-size team, and only the utilities of active agents shape Q_{tot} and receive updates. Even though this implementation respects the CTDE paradigm and the QMIX factorization it still had one fatal flaw that went unseen until the projects conclusion: masking off the gradient blinded the agents from seeing any usefulness of their self-sacrifice on the trap. Even if the mixer and utility networks could agree on how to handle traps in the end the masking of the gradients so that the agents never saw anything blinded them to the usefulness of this action and would essentially force their utility network to disagree and made training even more noisy. The lesson here is that ghosts can still see. The other problem with the padding we did was that padded and masked algorithms when tested on a different maze with $K - n$ agents suffered significant state-space distribution shifts. Even if the algorithms could overcome the training hurdles it is entirely likely that the networks would not gracefully generalize

like this, and a solution besides training on many different mazes and testing on a new unique maze may be required.

5 Experiments

We had no real baseline algorithms and so we evaluated QMIX directly against QMIX. Our experimental results are extremely preliminary and any results we gathered were artifacts of development and bug fixing, and the results are given as-is.

5.1 Experimental Setup and Training

For our experimental setup we generated two independent 7×7 mazes from Wilsons algorithm. We defined a tuneable trap mean and found 0.05 to 0.1 was more than sufficient. In most of the experimentation towards the end of the project this mean was set to 0. The full team size K was set to 10 and n to 2, so the team sizes of the full sized algorithm were 10 and the reduced team size 8. A full and reduced team was trained on each of the two mazes and then not only swapped but the full teams were cut down to 8 agents by padding out 2 of the agents.

Training length was significant for some of our testing and the only signs of success came from 100000 episode trained algorithms that performed reasonably well in training, but still failed during evaluation. Each episode was capped at 250 steps, though this appeared to be far too many steps for a 7×7 maze. ϵ was set to decay at a rate of 0.05 per 1000 episodes in the longest training run. In many shorter runs we set the decay to be very aggressive, which yielded very poor results indicating that QMIX takes a significant amount of exploration time.

5.2 Baselines

A completely random policy $\epsilon = 1.0$ was implemented as a sanity check to demonstrate that the environment transition and agent actions weren't fundamentally misbehaving. The random policy produced walks that proved the maze was at least navigable, and the goal could be reached and give reward. We did not compare QMIX to the random walk, however if we did the random walk would vastly outperform our QMIX training attempts, as random walks were marginally successful every time.

5.3 Results

Shown in Figure 1 here is the heavily shortened and aggressive ϵ decay training regime, where ϵ decayed by 0.05 every 50 episodes. This is not the configuration that should be used for actual testing runs but was routinely used during development to check specific implementation level development along with debug

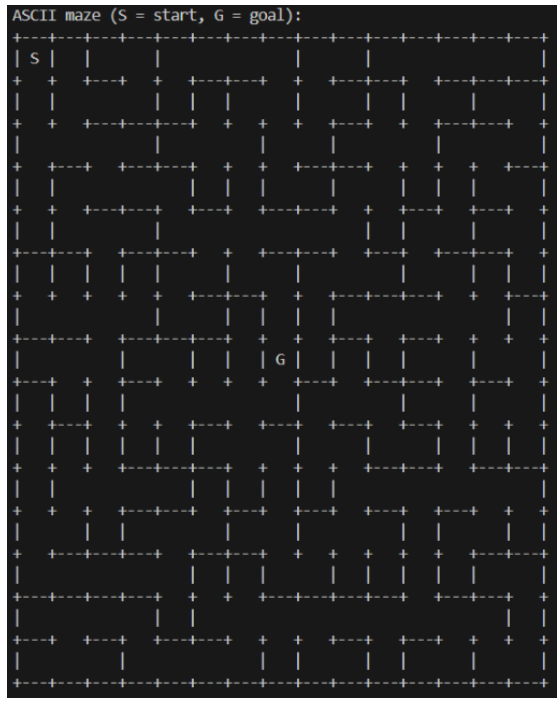

Figure 1: 7×7 Wilson Maze

outputs.

Training and results were largely unsuccessful across the board. Very low mean success rate across training compared to team sizes and in the single run evaluations there was almost never a single success although occasionally we had some. With the most recent and final version of the code with many of the fixes previously mentioned implemented a relatively short training period of 1500 episodes yielded success for algorithms trained with and tested with the same number of agents on their "home" maze. For this we did reintroduce a small negative step cost, but kept the distance based oracle shaped reward. It is very likely that if the project is in a state where it can learn it would take significant investigation into the appropriate combination of rewards and penalties.

This initial success in Figure 2 demonstrates the plausibility of our experiment and proves QMIX will learn path-finding algorithms. In general however it was very difficult to distinguish if the failure of the algorithm was the many implementation problems we had, the strong assumptions we made, or fundamental irreconcilable behavior of QMIX itself being unable to perform. This ambiguity is still very interesting and although our results were ambiguous and inconclusive they do not naturally demonstrate any critical issues with the ex-

```python
# QMIX learner


@dataclass
class QMIXConfig:
    n_agents: int
    obs_dim: int
    state_dim: int
    n_actions: int = 5
    hidden_dim: int = 64
    mixer_embed_dim: int = 32
    gamma: float = 0.99
    lr: float = 5e-4
    batch_size_eps: int = 8
    target_update_interval: int = 200
    grad_clip_norm: float = 10.0
    epsilon_start: float = 1.0
    epsilon_end: float = 0.05
    epsilon_decay_episodes: int = 50
```

Figure 2: DEBUG Hyperparameters

```
[Train] Episode 95000/100000, last loss = 0.0430
[Train] Episode 96000/100000, last loss = 0.0272
[Train] Episode 97000/100000, last loss = 0.0320
[Train] Episode 98000/100000, last loss = 0.0046
[Train] Episode 99000/100000, last loss = 0.0035
[Train] Episode 100000/100000, last loss = 0.0030
Results:
m1_k_train__m2_kn_test {'success_rate': 0.0, 'mean_survivors': 2.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m2_k_train__m1_kn_test {'success_rate': 0.0, 'mean_survivors': 2.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m1_k_train__m1_k_test {'success_rate': 0.0, 'mean_survivors': 4.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m1_kn_train__m1_kn_test {'success_rate': 0.0, 'mean_survivors': 2.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m2_k_train__m2_k_test {'success_rate': 0.0, 'mean_survivors': 4.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m2_kn_train__m2_kn_test {'success_rate': 0.0, 'mean_survivors': 2.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
```

Figure 3: 100000 Episode Early Training Result

```
Results:
m1_k_train__m2_kn_test {'success_rate': 0.0, 'mean_survivors': 8.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m2_k_train__m1_kn_test {'success_rate': 0.0, 'mean_survivors': 8.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m1_k_train__m1_k_test {'success_rate': 1.0, 'mean_survivors': 10.0, 'mean_steps': 6.0, 'mean_deaths': 0.0}
m1_kn_train__m1_kn_test {'success_rate': 0.0, 'mean_survivors': 8.0, 'mean_steps': 250.0, 'mean_deaths': 0.0}
m2_k_train__m2_k_test {'success_rate': 1.0, 'mean_survivors': 10.0, 'mean_steps': 23.0, 'mean_deaths': 0.0}
m2_kn_train__m2_kn_test {'success_rate': 1.0, 'mean_survivors': 8.0, 'mean_steps': 25.0, 'mean_deaths': 0.0}
```

Figure 4: 1500 Episode Current Training Successes

periment, just that the initial construction of the experiment was too aggressive in many ways.

6 Conclusion

Our experiment made strong assumptions on QMIX, and how the environment would interact with QMIX to force generalization and non-monotonic tradeoffs without forcing total failure. Our main assumptions were as follows:

- The mixer network can generalize gracefully even with padded agents.
- Non-monotonic tradeoffs can be made as long as the global policy demands it.
- QMIX could learn a path-finding algorithm or policy.

As discussed padding and masking was our way of dealing with the implementation side of dynamic team sizes. The mixer seemed to be able to handle this during training but it is likely that changing both the team size as well as the state encoding during evaluation causes outright failure. Non-monotonic tradeoffs seem plausible despite some of the mathematical nuance a strong enough team driven reward appears to be able to overcome this as well but this is speculation at best. The final assumption that QMIX can learn path-finding policies in partially observable settings is supported by [3] in literature discussed in section 3.3 but our initial training methodology of training only on one maze likely caused significant overfitting to that specific maze. Although never implemented we would have liked to implement a train on many mazes test on a sample maze from another teams training batch system but never had time. This would allow for policy to learn general maze path-finding behavior rather than a specific policy for the maze it was entirely trained in.

given more time and hindsight we would have liked to:

- Develop a mechanism to gradually introduce traps.
- Train on many mazes rather than just one.
- Maintain the swapping based evaluation but be more pointed in testing for policy generalization (new maze) or mixer generalization (missing agents).
- introduce a network to represent per-agent trajectories to assist QMIX's ability to learn pathfinding algorithms (policies) if QMIX significantly struggled to learn general maze pathfinding given that the algorithm was trained on many mazes but failed.

References

[1] Samvelyan, M., Rashid, T., Schroeder de Witt, C., Farquhar, G., Nardelli, N., Rudner, T. G. J., Hung, C.-M., Torr, P. H. S., Foerster, J. & Whiteson, S., "The StarCraft Multi-Agent Challenge," arXiv preprint arXiv:1902.04043, 2019.

[2] Stern, R., Sturtevant, N. R., Felner, A., Koenig, S., Ma, H., Walker, T. T., Li, J., Atzmon, D., Cohen, L., Kumar, T. K. S., Boyarski, E., & Barták, R. (2019). Multi-agent pathfinding: Definitions, variants, and benchmarks. Proceedings of the AAAI Conference on Artificial Intelligence, 33(01), 9837–9845

[3] V. Davydov, A. Skrynnik, K. S. Yakovlev & A. I. Panov, "Q-Mixing Network for Multi-Agent Pathfinding in Partially Observable Grid Environments," arXiv preprint arXiv:2108.06148, 2021.